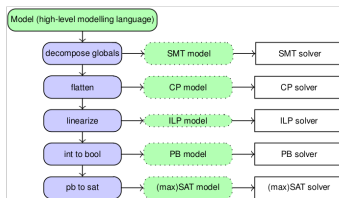


L06: Solving technologies and encodings, part 1



Prof. Tias Guns and Dr. Hendrik ‘Henk’ Bierlee

KU LEUVEN

Partly based on slides from Pierre Flener and H  l  ne Verhaeghe.

Recap: search, inference and primitive constraints

From Lecture 1:

Solving combinatorial problems **intelligently**:

- ▶ **Search**: Explore the space of possible assignments.
- ▶ **Inference**: Reduce the space to feasible (partial) assignments.

From Lecture 3:

Definition

Primitive constraint: a constraint that the solver can do **inference** with.

For CP, PB and SAT **inference=propagation**.

Definition

Propagation: using the properties of a primitive constraint to **reduce the domains** of participating decision variables by removing unsupported values.

Example: $x + 2 \cdot y \geq 3$, $D(x) = \{0, 1, 2\}$, $D(y) = \{0, 1, 2\}$
 $\implies D(y) = \{\textcolor{red}{0}, 1, 2\}$

Recap: solvers and primitive constraints

Different solver paradigms have different **primitive** constraints:

- ▶ **Boolean SATisfiability solvers:** Clause: $b_1 \vee b_2 \vee \dots \vee b_n$
- ▶ **Pseudo-Boolean solvers:** $\sum w \cdot b \leq c$
(with w, c constant, $\leq \in \{=, <, \leq, >, \geq\}$)
- ▶ **Integer Linear solvers:** $\sum w \cdot v \leq c$ and $v \leq c$
(with w, c constant, $\leq \in \{=, \leq, \geq\}$)

CP solvers have all-of-the-above + $\neq$ + *global constraints*

How to Compare Solving Technologies?

Specification language:

- ▶ What types of decision variables are supported?
- ▶ What types of primitive constraints are supported?
- ▶ Can there be an objective function, or only satisfaction?

Features:

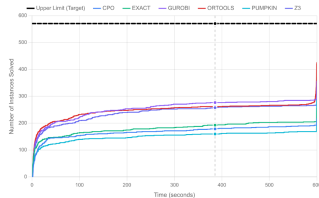
- ▶ Are its solvers **exact**, given enough time?
E.g. will they prove unsatisfiability? prove optimality?
- ▶ In which application areas has the technology been successfully used?
- ▶ Does the solving technology align well with this type of problem?

Typical goal: solve as **fast** as possible... but no silver bullet: NP-hard problems

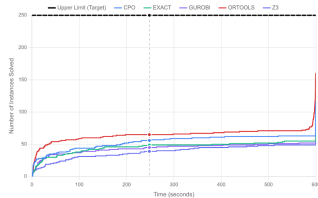
Efficient solving

How efficient will a solver be in solving your problem?

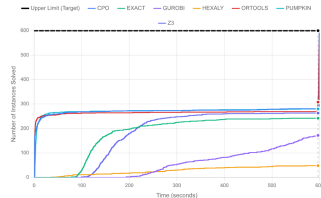
MaxSatEval-w '24



XCSP3-Opt '24



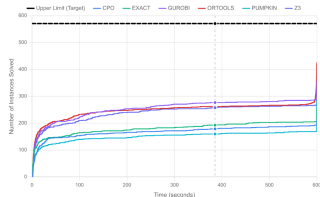
RCPSP J120



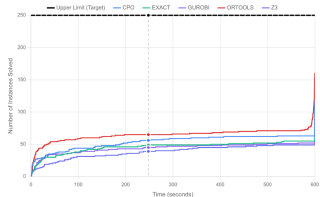
Efficient solving

How efficient will a solver be in solving your problem?

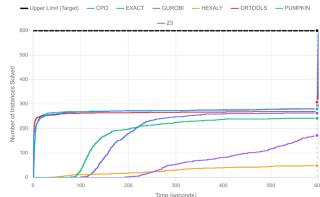
MaxSatEval-w '24



XCSP3-Opt '24



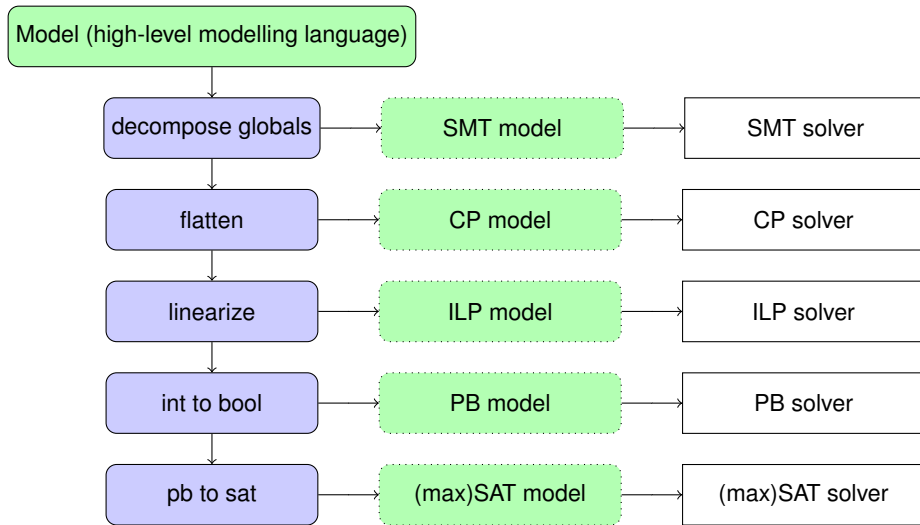
RCPSP J120



To get more insight, you need to understand

- ▶ What constraints are in your model?
- ▶ How close are these constraints to the primitive constraints of the solver (= how much *encoding* is needed)
- ▶ How well can the solver handle the resulting primitive constraints (= how strong do we expect the *inference* to be?)

From Model to Model to Solver: transformations



Goal of the lecture

Part 1 of solving technologies and encodings: SMT, decomposing global constraints, and CP solving (search + propagation).

Different solvers will behave differently for different problems...

Typical CP problems are NP-hard, and we can not expect to solve all problems efficiently (unless $P=NP$).

But many problems of practical interest:

- ▶ Can be practically solved
- ▶ Can be solved even faster with the right solving technology

Goal of the lecture

Part 1 of solving technologies and encodings: SMT, decomposing global constraints, and CP solving (search + propagation).

Different solvers will behave differently for different problems...

Typical CP problems are NP-hard, and we can not expect to solve all problems efficiently (unless $P=NP$).

But many problems of practical interest:

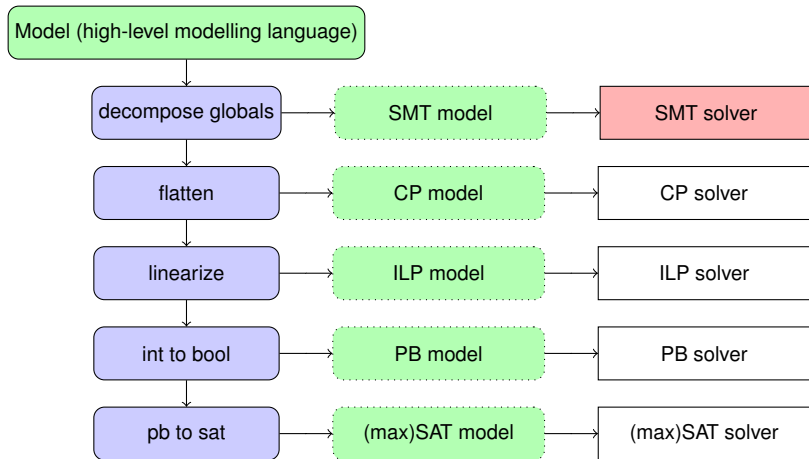
- ▶ Can be practically solved
- ▶ Can be solved even faster with the right solving technology

An overview of some solving technologies:

- ▶ to understand their advantages and limitations
- ▶ to help you choose a technology for a particular model
- ▶ to help you encode and adapt a model to a particular technology.

Next lecture (Part 2): flatten nested expressions; ILP, PB, and SAT encodings.

Overview



SMT: SAT Modulo Theories

Generalizes SAT solving beyond Boolean variables:

$$\text{SMT solver} = \text{SAT solver} + \text{Theory solver}$$

Used for many **verification** problems (prove a set of statements is True/False):

- ▶ Verification of hardware designs: Theory of Bit-Vectors
- ▶ Verification of computer programs: Theory of Arrays
- ▶ Verification of neural networks: Theory of Real Arithmetic
- ▶ Security analysis and input validation: Theory of Strings
- ▶ Transition systems, symbolic executing, planning: Theory of Integer Arithmetic

Many solvers support *theory combination* to combine multiple theories.

SMT theories

Definition

A *theory*

- ▶ defines theory-specific variables (called a *sort*) and valid **constraint predicates** over those;
- ▶ is associated with a theory-solver for any conjunction of its **predicates**.

SMT theories

Definition

A theory

- ▶ defines theory-specific variables (called a *sort*) and valid **constraint predicates** over those;
- ▶ is associated with a theory-solver for any conjunction of its **predicates**.

Example formula for the theory of Linear Integer Arithmetic (LIA):

$$((x < 3) \vee a)$$

$$(\neg(x \leq 4) \vee a \vee b)$$

$$(x \geq 0)$$

$$(x = y + 1) \vee (x \leq 2 \cdot y)$$

- ▶ **Sort:** integers (can be unbounded! unlike in CP) (*here: x and y*)
- ▶ **Constraint predicates:** **linear equalities and inequalities**
- ▶ **Theory-solver:** integer linear reasoning

Satisfiability Modulo Theories (SMT)

There is a standardized language for many different theories, that SMT solvers accept: the SMT-LIB language.

SMT determines the satisfiability of a logical formula over (one or more) background theories.

If SAT: return SAT + a solution to the *theory* problem

If UNSAT: return UNSAT.

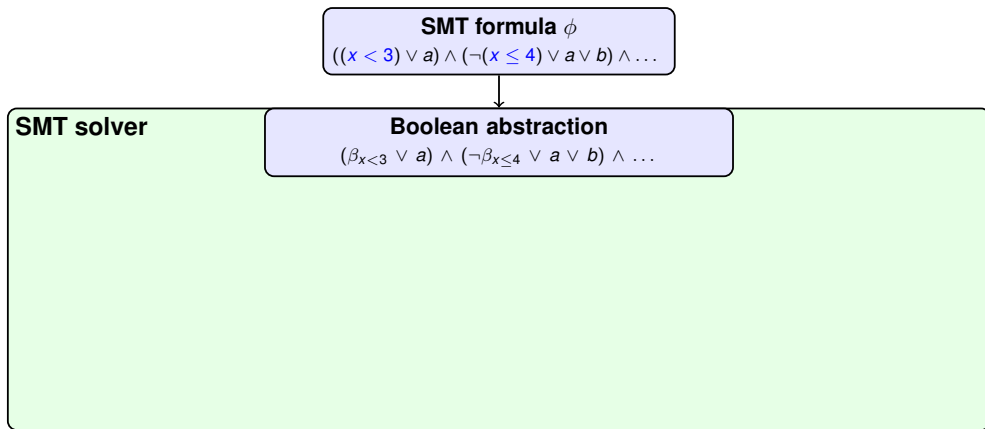
Lazy SMT Solving Architecture¹

SMT formula ϕ

$((x < 3) \vee a) \wedge (\neg(x \leq 4) \vee a \vee b) \wedge \dots$

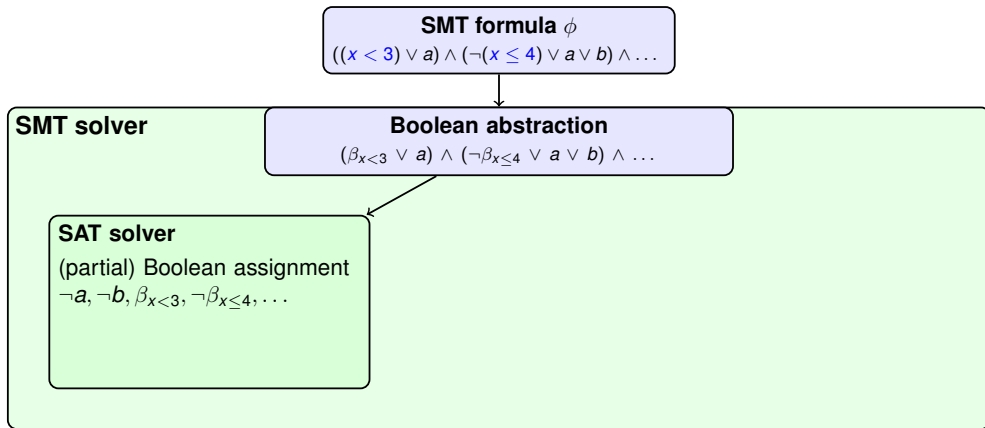
¹From: <https://users.aalto.fi/~tjunttil/2020-DP-AUT/notes-smt/lazy.html>

Lazy SMT Solving Architecture¹



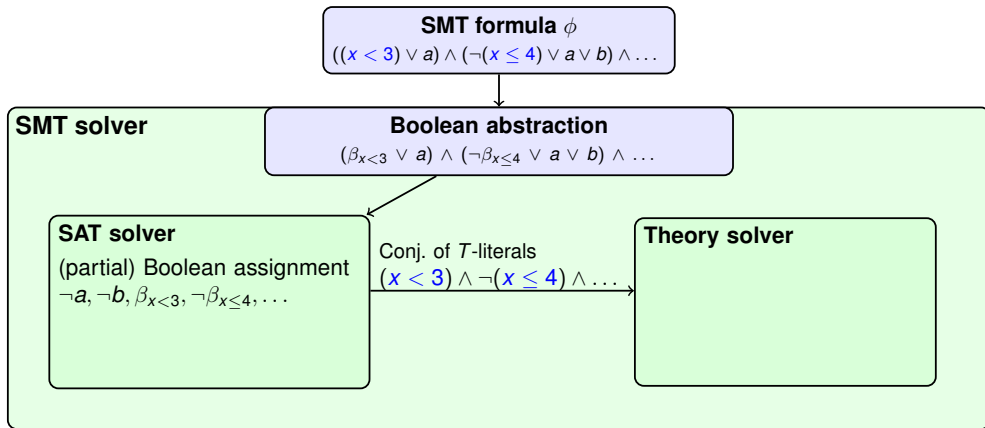
¹From: <https://users.aalto.fi/~tjunttil/2020-DP-AUT/notes-smt/lazy.html>

Lazy SMT Solving Architecture¹



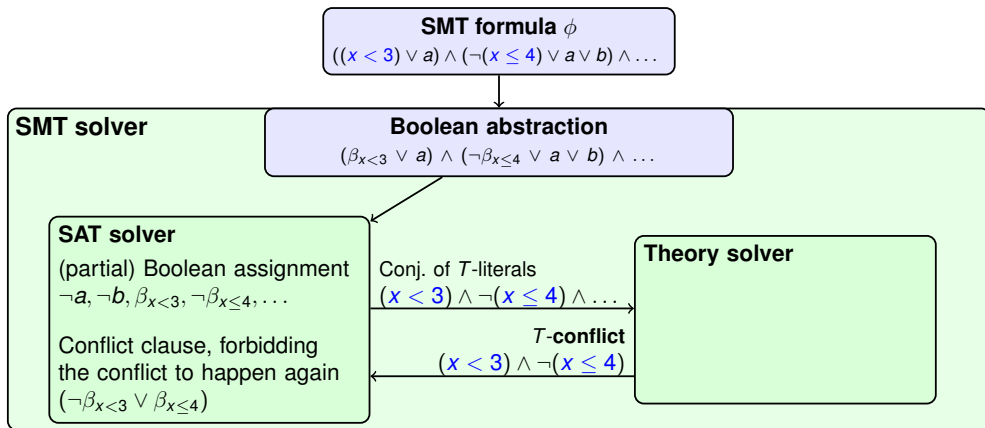
¹From: <https://users.aalto.fi/~tjunttil/2020-DP-AUT/notes-smt/lazy.html>

Lazy SMT Solving Architecture¹



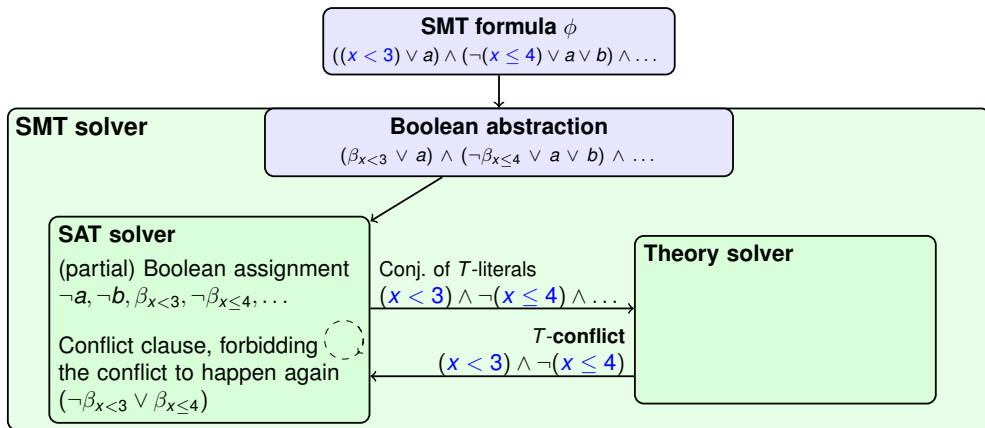
¹From: <https://users.aalto.fi/~tjunttil/2020-DP-AUT/notes-smt/lazy.html>

Lazy SMT Solving Architecture¹



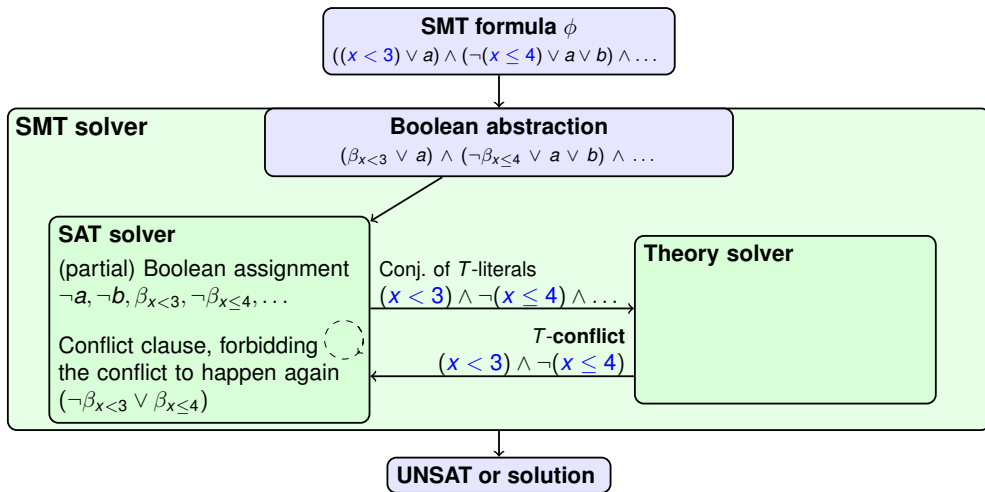
¹From: <https://users.aalto.fi/~tjunttil/2020-DP-AUT/notes-smt/lazy.html>

Lazy SMT Solving Architecture¹



¹From: <https://users.aalto.fi/~tjunttil/2020-DP-AUT/notes-smt/lazy.html>

Lazy SMT Solving Architecture¹



¹From: <https://users.aalto.fi/~tjunttil/2020-DP-AUT/notes-smt/lazy.html>

Lazy SMT Solving Architecture

Key idea: combine Boolean level reasoning with theory-specific algorithms.

Boolean abstraction:

- ▶ Separates the theory constraints from the logic: each theory constraint is represented by a T-literal² in the SAT solver.
- ▶ The result is a pure logic formula over literals (the Boolean abstraction)

SAT solver:

- ▶ Handles the logic part, including disjunctions of theory constraints
- ▶ Sends valid (partial) assignments of T-literals to the theory solver for checking

²if ' a ' is a Boolean decision variable, then both ' a ' and ' $\neg a$ ' are literals

Lazy SMT Solving Architecture

Key idea: combine Boolean level reasoning with theory-specific algorithms.

Boolean abstraction:

- ▶ Separates the theory constraints from the logic: each theory constraint is represented by a T-literal² in the SAT solver.
- ▶ The result is a pure logic formula over literals (the Boolean abstraction)

SAT solver:

- ▶ Handles the logic part, including disjunctions of theory constraints
- ▶ Sends valid (partial) assignments of T-literals to the theory solver for checking

Theory solver:

- ▶ Receives *conjunctions* of T-literals/theory constraints, and checks their validity
- ▶ If they are theory-satisfiable, can return a solution to the theory variables
- ▶ If not satisfiable, will return a T-conflict: a subset of T-literals that can not be satisfied together

²if ' a ' is a Boolean decision variable, then both ' a ' and ' $\neg a$ ' are literals

Lazy SMT Solving Architecture, efficiently

Additional techniques to make it more efficient:

- ▶ The SAT solver will pass T-literals to the theory solver one by one, as soon as it derives them (e.g. through propagation or branching)
- ▶ The theory solver is **incremental**: it can incrementally check only the relevant parts of the theory when a T-literal is set

Lazy SMT Solving Architecture, efficiently

Additional techniques to make it more efficient:

- ▶ The SAT solver will pass T-literals to the theory solver one by one, as soon as it derives them (e.g. through propagation or branching)
- ▶ The theory solver is **incremental**: it can incrementally check only the relevant parts of the theory when a T-literal is set
- ▶ The theory solver tries to return as small a conflict as efficiently possible (helps the SAT solver)

Lazy SMT Solving Architecture, efficiently

Additional techniques to make it more efficient:

- ▶ The SAT solver will pass T-literals to the theory solver one by one, as soon as it derives them (e.g. through propagation or branching)
- ▶ The theory solver is **incremental**: it can incrementally check only the relevant parts of the theory when a T-literal is set
- ▶ The theory solver tries to return as small a conflict as efficiently possible (helps the SAT solver)
- ▶ The theory solver can be asked for theory-implied literals; for which it can return a clausal explanations (helps the SAT solver)

(e.g. $\beta_{y>2} : \beta_{x>2} \wedge \beta_{y>x} \rightarrow \beta_{y>2}$, *equivalently* $\neg\beta_{x>2} \vee \neg\beta_{y>x} \vee \beta_{y>2}$)

SMT/OMT for combinatorial problems

Theory: QF_LIA = "Quantifier Free, Linear Integer Arithmetic" (and QF_NIA in case of non-linearities; theory combination comes at a small cost)

Supports Bool and Int, as well as logical and arithmetic operators; including nested expressions thereof.

No global constraints / global functions in QF_LIA:

- ▶ requires to **decompose** global constraints

SMT/OMT for combinatorial problems

Theory: QF_LIA = "Quantifier Free, Linear Integer Arithmetic" (and QF_NIA in case of non-linearities; theory combination comes at a small cost)

Supports Bool and Int, as well as logical and arithmetic operators; including nested expressions thereof.

No global constraints / global functions in QF_LIA:

- ▶ requires to **decompose** global constraints

OMT (Optimisation Modulo Theories) is an extension for optimisation supported by some SMT solvers

SMT/OMT for combinatorial problems

Theory: QF_LIA = "Quantifier Free, Linear Integer Arithmetic" (and QF_NIA in case of non-linearities; theory combination comes at a small cost)

Supports Bool and Int, as well as logical and arithmetic operators; including nested expressions thereof.

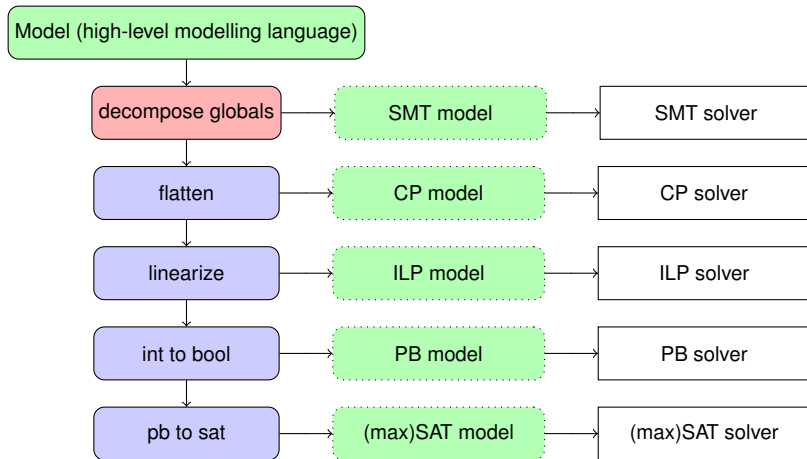
No global constraints / global functions in QF_LIA:

- ▶ requires to **decompose** global constraints

OMT (Optimisation Modulo Theories) is an extension for optimisation supported by some SMT solvers

Unlike finite-domain CP solvers, SMT solvers are not restricted to finite-domain problems (but this unused generality can have some efficiency cost).

Overview



Decomposition of unsupported globals

For **global constraints**: replace it by its decomposition.

Example: $\text{ALLDIFFERENT}(u, v, w) \implies u \neq v \wedge u \neq w \wedge v \neq w$

For **global functions**, used in a numeric expression:

Decomposition of unsupported globals

For **global constraints**: replace it by its decomposition.

Example: $\text{ALLDIFFERENT}(u, v, w) \implies u \neq v \wedge u \neq w \wedge v \neq w$

For **global functions**, used in a numeric expression:

- ▶ The classic view is that you can replace the global function by a fresh auxiliary variable, and add the functional global constraint, or rather its decomposition, over the auxiliary as a new constraint.

Example: $\text{COUNT}(X, 0) \geq 2$

$$\equiv a \geq 2 \wedge \text{COUNT}_{\text{Eq}}(X, 0, a)$$

$$\equiv a \geq 2 \wedge a = \sum_i \llbracket X_i = 0 \rrbracket$$

Decomposition of unsupported globals

For **global constraints**: replace it by its decomposition.

Example: $\text{ALLDIFFERENT}(u, v, w) \implies u \neq v \wedge u \neq w \wedge v \neq w$

For **global functions**, used in a numeric expression:

- We can alternatively separate a global function decomposition into a 'value' expression and a set of 'defining' constraints. We then replace the global function by the 'value' expression, and add the 'defining' constraints as new constraints.

Example 1: $\text{COUNT}(X, 0) \geq 2 \equiv \sum_i \llbracket X_i = 0 \rrbracket \geq 2$ (no defining constraints)

Example 2: $\text{NVALUE}(X) \geq 2 \equiv \sum_v B_v \geq 2 \wedge \forall v \in V : B_v = \bigvee_i (X_i = v)$

Decomposition and negation

If we also want to support **negating global constraints**, then we have to be more careful!

If a **global constraint** decomposition introduces new auxiliary variables, we can not simply negate the decomposition.

Example

Imagine a global constraint $\text{EQUALS2}(x)$ with decomposition $x = a \wedge a = 2$, e.g. it uses an auxiliary variable to ensure x equals 2.

If we have constraint $\neg \text{EQUALS2}(x)$ we can not simply replace the subexpression by its decomposition.

If we would, we would get $\neg(x = a \wedge a = 2)$ which equals $(x \neq a) \vee (a \neq 2)$ hence $x = 2$ would be a valid solution as a can just take any value other than 2.

Decomposition and negation

If we also want to support **negating global constraints**, then we have to be more careful!

If a **global constraint** decomposition introduces new auxiliary variables, we can not simply negate the decomposition.

A solution is also for global constraints to separate its decomposition into a (Boolean) '**value**' expression and a set of '**defining**' constraints.

Then we must do the following:

1. we replace the global constraint by the **value expression**
2. we add the **defining constraints** directly to the solver (always enforced)

In our example case $\text{EQUALS2}(x) \equiv x = a \wedge a = 2$,

So then $(y \vee \neg \text{EQUALS2}(x))$

$$\equiv (y \vee \neg(x = a)) \wedge a = 2$$

For **global functions** we also need to be careful if it is a **partial function** AND if it is used within a reified expression or negation.

A global function is **partial** if its value is undefined for some assignment to its decision variables.

The following 3 global functions can be partial:

- ▶ Integer division x/y : undefined when $y = 0$
- ▶ Modulo $x \bmod y$: undefined when $y = 0$
- ▶ Element $Arr[idx]$: undefined when idx is not in the range of Arr

For **global functions** we also need to be careful if it is a **partial function** AND if it is used within a reified expression or negation.

A global function is **partial** if its value is undefined for some assignment to its decision variables.

The following 3 global functions can be partial:

- ▶ Integer division x/y : undefined when $y = 0$
- ▶ Modulo $x \bmod y$: undefined when $y = 0$
- ▶ Element $Arr[idx]$: undefined when idx is not in the range of Arr

A solution is to transform the reified/negated partial function into a modified reified/negated *safened* total function, following the *relational semantics*.

Fine details: *Alan Frisch and Peter J. Stuckey. "The proper treatment of undefinedness in constraint languages." CP 2009,*

Implementation: <https://cpmpy.readthedocs.io/en/latest/api/transformations/safening.html>

Decomposition, example

Rewrite global constraints *that the solver does not support*, using (more) primitive constraints. For SMT: all globals.

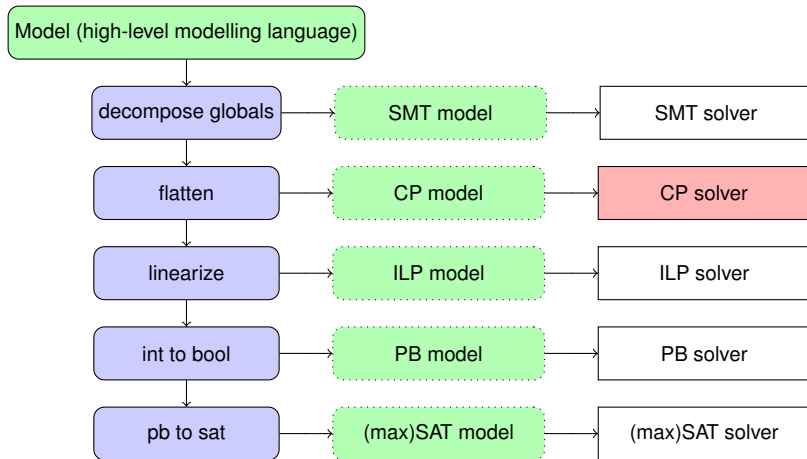
Example constraint model:

$$\begin{aligned} & \text{ALLDIFFERENT}(x, y, z) \\ & (x = 3) \rightarrow (y + z \leq 3) \\ & x, y, z \in \{1, 2, 3\} \end{aligned}$$

After transformations:

$$\begin{aligned} & x \neq y \wedge x \neq z \wedge y \neq z \\ & (x = 3) \rightarrow (y + z \leq 3) \\ & x, y, z \in \{1, 2, 3\} \end{aligned}$$

Overview



CP: Constraint Programming

For *finite-domain* combinatorial search problems:

$$\text{CP} = \text{propagation} + \text{search}$$

Includes logical, arithmetic and specialised *global constraints*. Solves both satisfaction and optimisation problems.

Typical Applications

- ▶ Scheduling, Timetabling, Assignment problems
- ▶ Routing Problems, Packing problems, esp. with side-constraints
- ▶ Puzzles and Games, Configuration Problems

Domains

Central concept: the **domain store**

Definition

The **domain** of a decision variable v , denoted by $\text{dom}(v)$, is the set of values that v can still take during **search**:

- ▶ A domain can only be **reduced**, by **search** and **propagation**
- ▶ A decision variable is said to be **fixed** if its domain is a singleton.
- ▶ **Unsatisfiability** occurs if the domain of a decision variable becomes empty, search then backtracks (and restores the domains)

Domains

Central concept: the **domain store**

Definition

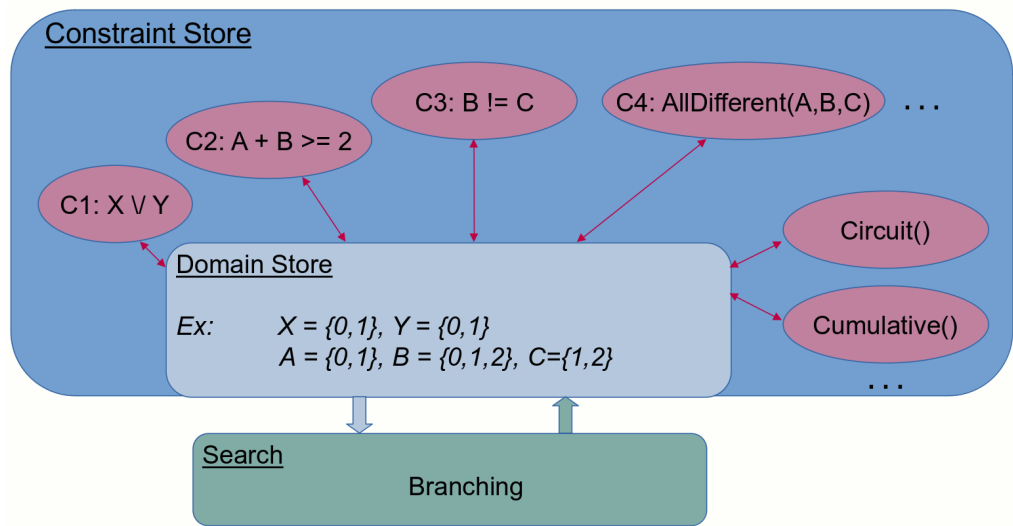
The **domain** of a decision variable v , denoted by $\text{dom}(v)$, is the set of values that v can still take during **search**:

- ▶ A domain can only be **reduced**, by **search** and **propagation**
- ▶ A decision variable is said to be **fixed** if its domain is a singleton.
- ▶ **Unsatisfiability** occurs if the domain of a decision variable becomes empty, search then backtracks (and restores the domains)

Each individual constraint will **propagate** independently: it can read from the domain store, and reduce domains in the domain store.

Recap example: $x + 2 \cdot y \geq 3$, $D(x) = \{0, 1, 2\}$, $D(y) = \{0, 1, 2\}$
 $\implies D(y) = \{\cancel{0}, 1, 2\}$

CP solver architecture, propagator view



CP solver architecture, search tree view

- ▶ Initialize the domain store with the initial domain

³This creates a binary search tree, one can also branch over each value in $D(V)$ in turn

CP solver architecture, search tree view



- ▶ Initialize the domain store with the initial domain
- ▶ Each constraint **propagator** gets the current domain as input, and outputs the same or a *reduced* domain

³This creates a binary search tree, one can also branch over each value in $D(V)$ in turn

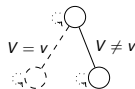
CP solver architecture, search tree view



- ▶ Initialize the domain store with the initial domain
- ▶ Each constraint **propagator** gets the current domain as input, and outputs the same or a *reduced* domain
- ▶ If all propagators are at **fixpoint** (their output=their input): create a **branch** in the search tree by choosing a non-fixed variable V and choosing a value $v \in D(V)$, and fixing the variable to this value: $D(V) = v$.

³This creates a binary search tree, one can also branch over each value in $D(V)$ in turn

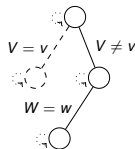
CP solver architecture, search tree view



- ▶ Initialize the domain store with the initial domain
- ▶ Each constraint **propagator** gets the current domain as input, and outputs the same or a *reduced* domain
- ▶ If all propagators are at **fixpoint** (their output=their input): create a **branch** in the search tree by choosing a non-fixed variable V and choosing a value $v \in D(V)$, and fixing the variable to this value: $D(V) = v$.
- ▶ On **backtracking** over this choice, restore to the previous $D(V)$ and enforce $V \neq v$ to explore the other branch³.

³This creates a binary search tree, one can also branch over each value in $D(V)$ in turn

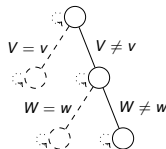
CP solver architecture, search tree view



- ▶ Initialize the domain store with the initial domain
- ▶ Each constraint **propagator** gets the current domain as input, and outputs the same or a *reduced* domain
- ▶ If all propagators are at **fixpoint** (their output=their input): create a **branch** in the search tree by choosing a non-fixed variable V and choosing a value $v \in D(V)$, and fixing the variable to this value: $D(V) = v$.
- ▶ On **backtracking** over this choice, restore to the previous $D(V)$ and enforce $V \neq v$ to explore the other branch³.
- ▶ If a propagator returns an empty domain for a decision variable: **fail and backtrack**

³This creates a binary search tree, one can also branch over each value in $D(V)$ in turn

CP solver architecture, search tree view



- ▶ Initialize the domain store with the initial domain
- ▶ Each constraint **propagator** gets the current domain as input, and outputs the same or a *reduced* domain
- ▶ If all propagators are at **fixpoint** (their output=their input): create a **branch** in the search tree by choosing a non-fixed variable V and choosing a value $v \in D(V)$, and fixing the variable to this value: $D(V) = v$.
- ▶ On **backtracking** over this choice, restore to the previous $D(V)$ and enforce $V \neq v$ to explore the other branch³.
- ▶ If a propagator returns an empty domain for a decision variable: **fail and backtrack**
- ▶ If all variables are *fixed* (have a singleton value): **return** the found solution.

³This creates a binary search tree, one can also branch over each value in $D(V)$ in turn

Propagation of constraints, more formally

Definition

A **propagator** for a constraint γ deletes from the domains of the decision variables of a γ -constraint, the values that cannot be in a solution to that constraint.

Examples

- ▶ For $x < y$: when $\text{dom}(x) = \{1..4\}$ and $\text{dom}(y) = \{-1..3\}$, delete $\{3, 4\}$ from $\text{dom}(x)$ and $\{-1..1\}$ from $\text{dom}(y)$.
- ▶ For $\text{ALLDIFFERENT}(x, y, z)$: when $\text{dom}(x) = \{1, 3\} = \text{dom}(y)$ and $\text{dom}(z) = \{1..4\}$, delete 1 and 3 from $\text{dom}(z)$ so that it becomes the non-range $\{2, 4\}$.

Propagation strength

The more values a propagator can remove, the fewer search we have to do, but propagation also takes computation time!

Propagation strength

The more values a propagator can remove, the fewer search we have to do, but propagation also takes computation time!

The propagation strength of a propagator can be expressed in terms of the **consistency** that it achieves:

Definition

- ▶ Arc consistency, for constraints between two variables A and B:
For each value v in A, there is a value w in B such that the constraint is satisfied, and vice-versa.
- ▶ Generalised arc consistency (GAC), or domain consistency:
every value in every variable exists in at least one solution to the constraint.
- ▶ Bounds consistency: the upper and lower bounds of every variable exist in at least one solution to the constraint.

Propagation strength

The more values a propagator can remove, the fewer search we have to do, but propagation also takes computation time!

The propagation strength of a propagator can be expressed in terms of the **consistency** that it achieves:

Definition

- ▶ Arc consistency, for constraints between two variables A and B:
For each value v in A, there is a value w in B such that the constraint is satisfied, and vice-versa.
- ▶ Generalised arc consistency (GAC), or domain consistency:
every value in every variable exists in at least one solution to the constraint.
- ▶ Bounds consistency: the upper and lower bounds of every variable exist in at least one solution to the constraint.

(there exist more forms of consistency, but GAC/Bounds are the most important)

Propagation strength

Example (Linear equality constraints)

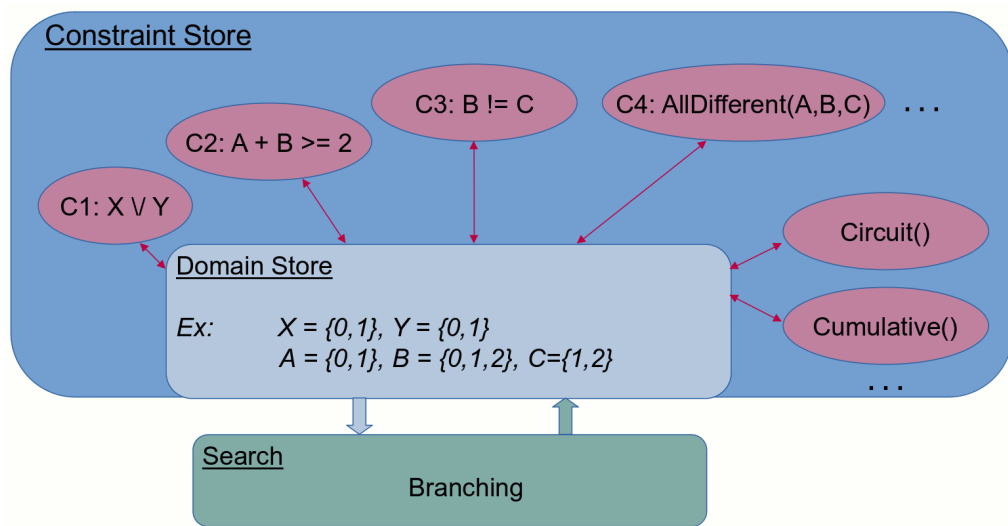
Consider the linear constraint $3 \cdot x + 4 \cdot y = z$
with $\text{dom}(x) = \{0, 1\} = \text{dom}(y)$ and $\text{dom}(z) = \{0, \dots, 10\}$:

- ▶ A bounds consistent propagator reduces $\text{dom}(z)$ to $\{0, \dots, 7\}$.
- ▶ A GAC propagator reduces $\text{dom}(z)$ to $\{0, 3, 4, 7\}$.

Time complexity:

- ▶ A bounds consistent propagator for a linear equality constraint can be implemented to run in $\mathcal{O}(n)$ time, where n is the number of decision variables in the constraint.
- ▶ A GAC propagator for a linear equality constraint can be implemented to run in $\mathcal{O}(n \cdot d^2)$ time, where n is the number of decision variables in the constraint and d is the sum of their domain sizes.

CP solver architecture, efficiency?

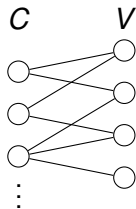


Efficient CP solvers, fixpoint algorithm

Avoiding propagator calls in the fixpoint propagation loop.



Efficient CP solvers, fixpoint algorithm

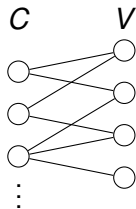


Avoiding propagator calls in the fixpoint propagation loop.

Only wake up when one of your variables changed

- ▶ Each variable has a *watch-list* of constraints that 'watch' it for domain change, only those are added to the propagator queue, until the queue is empty

Efficient CP solvers, fixpoint algorithm



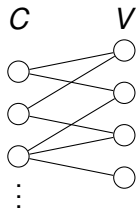
Avoiding propagator calls in the fixpoint propagation loop.

Only wake up when one of your variables changed (meaningfully)

- ▶ Each variable has a *watch-list* of constraints that 'watch' it for domain change, only those are added to the propagator queue, until the queue is empty
- ▶ In many solvers, propagators can register for specific domain events (e.g. only bound(s) or only when a variable is fixed)

Example: the $X \neq Y$ constraint can only propagate when a variable is fixed.

Efficient CP solvers, fixpoint algorithm

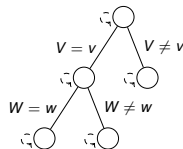


Avoiding propagator calls in the fixpoint propagation loop.

Only wake up when one of your variables changed (meaningfully)

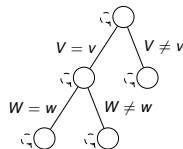
- ▶ Each variable has a *watch-list* of constraints that 'watch' it for domain change, only those are added to the propagator queue, until the queue is empty
- ▶ In many solvers, propagators can register for specific domain events (e.g. only bound(s) or only when a variable is fixed)
Example: the $X \neq Y$ constraint can only propagate when a variable is fixed.
- ▶ Often the queue has buckets so that 'cheaper to propagate' constraints are checked before more expensive ones
(because the sooner you check it, the higher the chance it is re-added to the queue when one of its variables change)

Efficient CP solvers, incremental state



Each propagator is **incremental**: only checks what has changed, and can keep its own (backtrackable) state.

Efficient CP solvers, incremental state

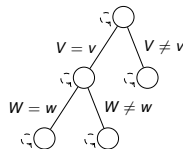


Each propagator is **incremental**: only checks what has changed, and can keep its own (backtrackable) state.

Efficient backtracking:

before branching, make a '**save point**' of the domains and propagator states, and restore to that save point on backtracking.

Efficient CP solvers, incremental state



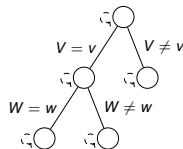
Each propagator is **incremental**: only checks what has changed, and can keep its own (backtrackable) state.

Efficient backtracking:

before branching, make a '**save point**' of the domains and propagator states, and restore to that save point on backtracking.

- Copying: copy the state before making a change, replace the original on backtracking

Efficient CP solvers, incremental state



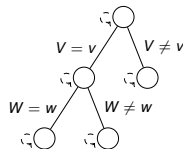
Each propagator is **incremental**: only checks what has changed, and can keep its own (backtrackable) state.

Efficient backtracking:

before branching, make a '**save point**' of the domains and propagator states, and restore to that save point on backtracking.

- ▶ Copying: copy the state before making a change, replace the original on backtracking
- ▶ Trailing (most solvers): keep a list of restoring operations: when making a change, record how to undo the change; on backtracking, undo the changes

Efficient CP solvers, incremental state



Each propagator is **incremental**: only checks what has changed, and can keep its own (backtrackable) state.

Efficient backtracking:

before branching, make a '**save point**' of the domains and propagator states, and restore to that save point on backtracking.

- ▶ Copying: copy the state before making a change, replace the original on backtracking
- ▶ Trailing (most solvers): keep a list of restoring operations: when making a change, record how to undo the change; on backtracking, undo the changes

Lots of research goes into developing highly efficient, incremental propagators for global constraints!

Search Strategies

CP Search is almost always **depth-first search** (with efficient backtracking).

Branching decisions:

- ▶ Which decision variable to branch on?
- ▶ Which value to branch on first?

Search Strategies

CP Search is almost always **depth-first search** (with efficient backtracking).

Branching decisions:

- ▶ Which decision variable to branch on?
- ▶ Which value to branch on first?

Goal: explore the search space as efficiently as possible.

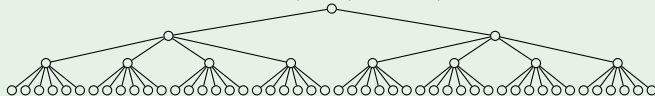
Can we *cheaply* estimate the quality of a branching decision? → **branching heuristics**

Variable Selection Heuristics: Min-Dom

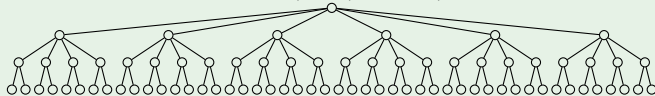
Example (Impact of domain size on search tree size)

Consider $x \in \{1, 2\}$, $y \in \{1, 2, 3, 4\}$, $z \in \{1, \dots, 6\}$

- If selecting the decision variables in the order x, y, z , the search tree has $1 + 2 + 2 \cdot 4 + 2 \cdot 4 \cdot 6 = \mathbf{59}$ nodes.



- If selecting the decision variables in the order z, y, x , the search tree has $1 + 6 + 6 \cdot 4 + 6 \cdot 4 \cdot 2 = \mathbf{79}$ nodes.



Variable Selection Heuristics: Min-Dom

Example (Impact of domain size on search tree size)

Consider $x \in \{1, 2\}$, $y \in \{1, 2, 3, 4\}$, $z \in \{1, \dots, 6\}$

- ▶ If selecting the decision variables in the order x, y, z , the search tree has $1 + 2 + 2 \cdot 4 + 2 \cdot 4 \cdot 6 = \mathbf{59}$ nodes.
- ▶ If selecting the decision variables in the order z, y, x , the search tree has $1 + 6 + 6 \cdot 4 + 6 \cdot 4 \cdot 2 = \mathbf{79}$ nodes.

Ignoring propagation, multi-way branching over variables with small domains higher in the tree leads to fewer internal search nodes.

Dom heuristic (select smallest): $|D(V)|$ (*cardinality/size of the set $D(V)$*)

Variable Selection Heuristics: Min-Dom/Deg

Degree of a decision variable V : the number of constraints that the variable is involved in.

Dom/Deg heuristic (select smallest): $\frac{|D(V)|}{\text{Deg}(V)}$

Idea: prefer small domains (smaller tree), but also variables that are involved in many constraints (higher chance to fail).

First-fail principle: if a decision leads to a fail, it is better to have it early in the search tree, to allow the pruning of many children nodes ⁴

⁴[Increasing tree search efficiency for CSPs, by R. Haralick and G. Elliott, IJCAI 1979]

Variable Selection Heuristics: Activity-based search

Can we capture how actively a decision variable is contributing to search space reductions?

Idea: propagation shrinks domains, hence recent domain changes indicate potentially important variables.

Variable activity with decay:

- ▶ Initialize an activity value $A(V)$ for each variable
- ▶ After propagation reaches fixed-point:
 - ▶ decay each activity by α : $A(X) = \alpha \cdot A(X)$
 - ▶ set $A(V) = A(V) + 1$ if $D(V)$ has been modified

Activity-based search heuristic (select largest): $\frac{A(V)}{|D(V)|}$

Variable Selection Heuristics: Conflict ordering search

Can we directly capture how often a variable is involved in a conflict?

Idea: if a branching decision results in propagation discovering a failure, then **timestamp** the variable branched on.

Conflict ordering search heuristic (select largest): $\text{timestamp}(V)$

Impact of search heuristics

Example

Finding the first solution to 250-queens with CPMpy-Choco (CP):

search	seconds
default	0.17
min-dom, lb value	0.19
conflict-ordering search	0.22
activity-based search	0.23
dom over (weighted)degree	> 30
input order, lb value	> 30

Note: value ordering heuristics also exist (e.g. lowerbound, upperbound, mid...)

Search heuristics in CP: historic perspective

Historically, constraint programming solvers allow programming your own constraints as well as **programmable search**:

Using your domain knowledge to determine variable/value ordering and choices, e.g. which variables, what branching decision are created... also full access to the constraint, domain store and solver statistics.

→ **CP as a generic depth-first search *framework***

Since a number of years, the CP community has been advancing on **black box** search strategies that work well in general (e.g. activity & conflict based search).

Search heuristics in CP: historic perspective

Historically, constraint programming solvers allow programming your own constraints as well as **programmable search**:

Using your domain knowledge to determine variable/value ordering and choices, e.g. which variables, what branching decision are created... also full access to the constraint, domain store and solver statistics.

→ **CP as a generic depth-first search *framework***

Since a number of years, the CP community has been advancing on **black box** search strategies that work well in general (e.g. activity & conflict based search).

→ **CP as generic satisfaction/optimisation *solver***

CP Solving, optimisation

CP is used for both satisfaction and optimisation problems.

In case of *optimisation problems* we assume the presence of an objective function $O(V)$ over a (subset of) the decision variables.

CP Solving, optimisation

CP is used for both satisfaction and optimisation problems.

In case of *optimisation problems* we assume the presence of an objective function $O(V)$ over a (subset of) the decision variables.

Most common technique: **solution-improving search**:

1. Find any satisfying solution
2. Compute the objective value o and add the solution-improving constraint $O(V) < o$ (in case of a minimisation problem)
3. Continue searching, repeat

CP Solving, optimisation

CP is used for both satisfaction and optimisation problems.

In case of *optimisation problems* we assume the presence of an objective function $O(V)$ over a (subset of) the decision variables.

Most common technique: **solution-improving search**:

1. Find any satisfying solution
2. Compute the objective value o and add the solution-improving constraint $O(V) < o$ (in case of a minimisation problem)
3. Continue searching, repeat

This will find a sequence of satisfying solutions and finally return UNSAT. The last solution found before that is an optimal solution.
(the solver proved no better solution exists)

Next lecture

How to flatten nested expressions to primitive CP constraints.

Lower-level, but highly optimized solver techniques:

- ▶ Integer Linear Programming
- ▶ Pseudo-Boolean solving
- ▶ Boolean Satisfiability (CDCL)